

Python

Списки. Работа с памятью



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос

?

Какие бывают изменяемые типы данных?

Изменяемые типы данных



Изменение в месте – позволяет модифицировать содержимое без создания нового объекта.



Ссылочная природа – передача в функцию или переменную создаёт ссылку на объект, а не его копию.



Эффективность – уменьшение нагрузки на память за счёт работы с одним объектом.



Методы изменения – возможность добавлять, удалять и изменять элементы внутри структуры.



Куча

Это область динамической памяти, в которой хранятся все объекты в Python.

Основные аспекты работы с кучей



Выделение памяти – автоматически выполняется интерпретатором Python.



Доступ по ссылке – переменные хранят ссылки, а не сами объекты.



Изменяемые и неизменяемые объекты – неизменяемые (int, str, tuple) создают новые объекты при изменении, изменяемые (list, dict, set) модифицируются на месте.



Сборщик мусора – Python автоматически очищает неиспользуемые объекты.



Функция id

Возвращает уникальный идентификатор объекта, который представляет его местоположение в памяти.

Пример использования id()



```
a = [1, 2, 3]
```

```
b = a
```

```
print(id(a)) # Идентификатор объекта a
```

```
print(id(b)) # Тот же идентификатор
```

```
c = [1, 2, 3]
```

```
print(id(c)) # Другой идентификатор, так как это другой объект
```

Экспресс-опрос

?

Как работает присваивание?

Присваивание неизменяемых объектов



Когда неизменяемый объект передаётся в другую переменную или функцию, Python копирует по ссылке.

Пример со строкой

```
text1 = "hello"

text2 = text1 # text2 ссылается на тот же объект

text1 + " Python" # Создаётся новый объект

print(text1)

text2 += " world" # Создаётся новый объект "hello world"

print(text1)

print(text2)
```



Присваивание изменяемых объектов



Когда изменяемый объект присваивается другой переменной, копирование происходит **по ссылке**.

Пример со строкой



```
list_a = [1, 2, 3]

list_b = list_a # list_b ссылается на тот же объект, что и list_a

# Добавление нового элемента

list_b.append(4) # Изменение объекта по ссылке

print(list_a)

print(list_b)

# Изменение элемента по индексу

list_b[0] = 'new'

print(list_a)

print(list_b)
```

Экспресс-опрос

?

Что такое вложенные коллекции?



Вложенные коллекции

Это коллекции, которые содержат другие коллекции в качестве элементов. Это позволяет строить более сложные структуры данных, такие как многомерные массивы, таблицы и другие.

Примеры вложенных коллекций



Список с числами и списками

```
list_elements = [[1, 2, 3], [4, 5], 6, [7], [8, 9]]
```

```
print(list_elements)
```

Кортеж со списками

```
lists_tuple = ([1, 2], [3, 4], [5, 6])
```

```
print(lists_tuple)
```

Список со списками

```
lists = [[1, 2], [3, 4]]
```

```
print(lists)
```

Экспресс-опрос

?

Что такое доступ к вложенным элементам?

Доступ к вложенным элементам



Пояснение

Для доступа к элементам вложенного списка используется несколько уровней индексации.

Вложенными элементами

```
list_elements = [[1, 2, 3], [4, 5], 6, [7, [8,  
[9], 10]]]
```

Доступ к вложенным элементам



Первого уровня

```
print(list_elements[0]) # [1, 2, 3]
print(list_elements[1]) # [4, 5]
print(list_elements[2]) # 6
print(list_elements[3]) # [7, [8, [9], 10]]
```

Вложенные списки

```
print(list_elements[0][1]) # 2
```

Доступ к вложенным элементам



Временные переменные

```
first_nested_list = list_elements[0]

second_element = first_nested_list[1]

print(second_element) # 2
```

Глубокая вложенность

```
print(list_elements[3][1][2]) # 10

print(list_elements[3][1][1][0]) # 9
```

Доступ к вложенным элементам



Изменения элементов

```
list_elements = [[1, 2], [3, 4], [5, 6]]
```

Замена элемента в списке

```
list_elements[0][1] = "two"

print(list_elements) # [[1, 'two'], [3, 4], [5, 6]]
```

Доступ к вложенным элементам



Замена вложенного списка

```
list_elements[2] = "new"

print(list_elements) # [[1, 'two'], [3, 4], 'new']
```

Строки внутри списка

```
list_elements[0][1] =
list_elements[0][1].upper()

print(list_elements) # [[1, 'TWO'], [3, 4], 'new']
```


Экспресс-опрос

?

Что такое итерация по вложенным коллекциям?

Первый уровень вложенности



Пояснение

Простая итерация по верхнему уровню коллекции.

Код

```
for sublist in list_elements:
    print(sublist)
```

Несколько уровней вложенности



Пояснение

Используем вложенные циклы `for`.

Код

```
for sublist in list_elements:
    for item in sublist:
        print(item, end=" ")
```

Итерация с изменением элементов



Пояснение

Изменяем элементы коллекции во время итерации.

Код

```
for i, sublist in enumerate(list_elements):
    for j, item in enumerate(sublist):
        list_elements[i][j] = item + 1

print(list_elements)
```

Экспресс-опрос

?

Для чего нужен оператор del?

Оператор del



Используется для удаления элементов коллекции по индексу, срезу, а также для удаления целых переменных.

Основные аспекты работы с кучей



Может удалять отдельные элементы, срезы или целые объекты.



Не возвращает значение удалённого элемента.



Освобождает память путём удаления переменных.



Попытка обращения к удалённому объекту вызывает `NameError`.

Синтаксис



```
del list[index] # Удаление одного элемента по индексу
```

```
del list[start:end] # Удаление среза элементов
```

```
del variable # Удаление переменной
```


Примеры



Удаление элемента по индексу

```
numbers = [10, 20, 30, 40]
```

```
del numbers[2]
```

```
print(numbers) # [10, 20, 40]
```

Удаление первого элемента

```
fruits = ["apple", "banana", "cherry"]
```

```
del fruits[0]
```

```
print(fruits) # ['banana', 'cherry']
```

Примеры



Удаление среза

```
numbers = [10, 20, 30, 40, 50]
```

```
del numbers[1:3]
```

```
print(numbers) # [10, 40, 50]
```

Удаление всех элементов (аналог `clear`)

```
numbers = [10, 20, 30, 40]
```

```
del numbers[:]
```

```
print(numbers) # []
```

Примеры



```
# Удаление переменной
```

```
old_numbers = [1, 2, 3]
```

```
new_numbers = old_numbers
```

```
del old_numbers
```

```
print(new_numbers) # [1, 2, 3]
```

Экспресс-опрос

?

Для чего нужно копирование списка?

Копирование списка



При копировании списков важно понимать, что присваивание списка другой переменной не создаёт новую копию. Вместо этого создаётся ссылка на тот же объект.

Пример присваивания



```
original_list = [1, 2, 3]
copied_list = original_list # Это не создаёт новую копию
copied_list[0] = 99
print(original_list) # [99, 2, 3]
```

Важно



При изменении `copied_list` изменяется и `original_list`, так как они ссылаются на один и тот же объект в памяти.

Экспресс-опрос

?

Что такое поверхностное и глубокое копирование?

Перегруженный интерфейс и хаотичность

Поверхностное копирование

Подходит для списков без вложенных структур или если вложенные элементы не будут изменяться.

Глубокое копирование

Рекомендуется для списков со вложенными объектами, когда нужно создать независимую копию на всех уровнях.

Поверхностное копирование



Создаёт новый список, копируя элементы из исходного списка, то есть ссылки на них. Если в списке есть вложенные списки или другие изменяемые объекты, то изменение этих вложенных объектов в копии также отразится на исходном списке.

Метод copy()



```
original_list = [1, 2, 3]
copied_list = original_list.copy()
copied_list[0] = 99
print(original_list)  # [1, 2, 3]
print(copied_list)   # [99, 2, 3]
```

Срез [:]



```
original_list = [1, 2, 3]
copied_list = original_list[:]
copied_list[0] = 99
print(original_list) # [1, 2, 3]
print(copied_list) # [99, 2, 3]
```

Функция list()



```
original_list = [1, 2, 3]
copied_list = list(original_list)
copied_list[0] = 99
print(original_list) # [1, 2, 3]
print(copied_list) # [99, 2, 3]
```

Копирование списка с вложенными объектами



```
original_list = [[1, 2], [3, 4]]  
  
shallow_copy = original_list.copy()  
  
shallow_copy[0][0] = 99  
  
print(original_list)  # [[99, 2], [3, 4]]  
  
print(shallow_copy)  # [[99, 2], [3, 4]]
```

Экспресс-опрос

?

Для чего нужно глубокое копирование?

Глубокое копирование



Создаёт новый список и полностью копирует все вложенные структуры, создавая независимые копии каждого вложенного объекта. Для этого используется функция `copy.deepcopy()` из модуля `copy`.

Пример глубокого копирования



```
import copy

original_list = [[1, 2], [3, 4]]

deep_copy = copy.deepcopy(original_list)

deep_copy[0][0] = 99

print(original_list)  # [[1, 2], [3, 4]]

print(deep_copy)     # [[99, 2], [3, 4]]
```

Экспресс-опрос

?

Что такое List comprehension?

List comprehension



Это удобный способ создать новый список, применяя выражение к каждому элементу существующего итерируемого объекта (например, списка, кортежа, строки) и/или фильтруя элементы по условию.

Синтаксис



```
new_list = [expression for item in iterable]
```



expression

Выражение (функция, операция или просто элемент), которое будет применено к каждому элементу. Результат будет добавлен в новый список.

Создание списка квадратов чисел



```
numbers = [1, 4, 6, 7, 9]

# Возведение каждого элемента numbers в квадрат

squares = [n ** 2 for n in numbers]

print(squares)

print(numbers) # Изначальный список останется без изменений
```

List comprehension и цикл for



Позволяют создавать и заполнять списки, но делают это по-разному. List comprehension обеспечивает более компактный и читаемый синтаксис, а цикл for предоставляет больше возможностей для сложной логики.

Вычисление квадратов чисел



Создание аналогичного списка с помощью List comprehension

```
print([x ** 2 for x in range(5)])
```

Эквивалент с циклом for

```
squares = []
```

```
for x in range(5):
```

```
    squares.append(x ** 2)
```

```
print(squares)
```


List comprehension и цикл for



Выражение `x ** 2` в List comprehension аналогично выражению `squares.append(x ** 2)` в цикле.

Преобразование верхний регистр



List comprehension

```
words = ["hello", "world", "python"]

uppercase_words = [word.upper() for word in words]

print(uppercase_words)
```

Эквивалент с циклом for

```
words = ["hello", "world", "python"]

uppercase_words = []

for word in words:

    uppercase_words.append(word.upper())

print(uppercase_words)
```

List comprehension с условием if



С условием if позволяет добавлять элементы в новый список только при выполнении определённого условия.

Синтаксис



```
new_list = [expression for item in iterable if condition]
```

Выбор только чётных чисел



```
# List comprehension
```

```
even_numbers = [x for x in range(10) if x % 2 == 0]
```

```
print(even_numbers)
```

```
# Эквивалент с циклом for
```

```
even_numbers = []
```

```
for x in range(10):
```

```
    if x % 2 == 0:
```

```
        even_numbers.append(x)
```

```
print(even_numbers)
```

Выбор слов, содержащих букву 'a'



List comprehension

```
words = ["apple", "banana", "cherry", "date"]

words_with_a = [word for word in words if 'a' in word]

print(words_with_a)
```

Эквивалент с циклом for

```
words = ["apple", "banana", "cherry", "date"]

words_with_a = []

for word in words:
    if 'a' in word:
        words_with_a.append(word)

print(words_with_a)
```

List comprehension с условием if-else



С условием `if...else` позволяет создавать новый список, где для каждого элемента применяется условие, и добавляется разное значение в зависимости от выполнения этого условия.

Синтаксис



```
new_list = [expression_if_true if condition else expression_if_false for item in iterable]
```


Замена нечётных чисел на -1



List comprehension

```
numbers = [2, 7, 5, 4, 1, 1, 7, 8]
modified_list = [x if x % 2 == 0 else -1 for x in numbers]
print(modified_list)
```

Эквивалент с циклом for

```
numbers = [2, 7, 5, 4, 1, 1, 7, 8]
modified_list = []

for x in numbers:
    if x % 2 == 0:
        modified_list.append(x)

    else:
        modified_list.append(-1)
print(modified_list)
```

Выбор слов, содержащих букву 'a'



List comprehension

```
words = ["cat", "elephant", "dog", "bird"]
modified_words = [word if len(word) > 3 else word.capitalize() for word in words]
print(modified_words)
```

Эквивалент с циклом for

```
words = ["cat", "elephant", "dog", "bird"]
modified_words = []

for word in words:
    if len(word) > 3:
        modified_words.append(word)
    else:
        modified_words.append(word.capitalize())

print(modified_words)
```

List comprehension с вложенным условием if-else



Поддерживает вложенные условия, позволяя добавлять несколько уровней проверки в одном выражении.

List comprehension



Пример

Замена слов на основе условий.

Задача

Если длина слова больше 5 символов, оставить его без изменений. Если длина слова от 3 до 5 символов, заменить слово на 'medium'. Если длина слова меньше 3 символов, заменить его на 'short'.

Выбор слов, содержащих букву 'a'



```
# List comprehension
words = ["hi", "apple", "banana", "cat", "blueberry", "on"]
modified_words = [word if len(word) > 5 else ('medium' if len(word) >= 3 else 'short') for word in words]
print(modified_words)

# Эквивалент с циклом for
words = ["hi", "apple", "banana", "cat", "blueberry", "on"]
modified_words = []
for word in words:
    if len(word) > 5:
        modified_words.append(word)
    else:
        if len(word) >= 3:
            modified_words.append('medium')
        else:
            modified_words.append('short')
print(modified_words)
```

List comprehension с вложенным условием if-else



Чаще всего list comprehension используются с простыми условиями, чтобы сохранить читаемость и простоту понимания кода.



Матрица

Это двумерная структура данных, представленная в виде вложенного списка (списка списков), где каждая строка содержит одинаковое количество элементов.

List comprehension с вложенным циклом



List comprehension поддерживает вложенные циклы, что позволяет создавать списки на основе более сложных структур данных, например, вложенных списков или матриц.

Синтаксис



```
new_list = [expression for item1 in iterable1 for item2 in iterable2]
```

Создание списка пар чисел



```
# List comprehension
```

```
pairs = [(x, y) for x in range(3) for y in range(2)]
```

```
print(pairs)
```

```
# Эквивалент с циклом for
```

```
pairs = []
```

```
for x in range(3):
```

```
    for y in range(2):
```

```
        pairs.append((x, y))
```

```
print(pairs)
```

Распаковка вложенных списков



```
# List comprehension
```

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
flattened = [num for row in matrix for num in row]
```

```
print(flattened)
```

```
# Эквивалент с циклом for
```

```
flattened = []
```

```
for row in matrix:
```

```
    for num in row:
```

```
        flattened.append(num)
```

```
print(flattened)
```

List comprehension



Преимущества

- Краткость и читаемость: Код выглядит более лаконичным и понятным для простых операций.
- Однострочное создание списка: Удобно для компактного представления и передачи результата в функции.

Недостатки

- Ограниченная читаемость для сложных условий: Если в выражении много логики или условий, его труднее читать.
- Не так гибко, как цикл for: Для более сложных операций и многошаговых вычислений цикл for более уместен.

Цикл for



Преимущества

- Гибкость: Подходит для выполнения сложных операций и добавления дополнительных условий и логики.
- Более лёгкая отладка: Код, написанный с использованием цикла `for`, легче понять и модифицировать при необходимости.

Недостатки

- Многословность: Требуется больше строк кода для выполнения простой операции по сравнению с `list comprehension`.

Экспресс-опрос

?

Для чего нужна функция zip?

Функция zip



Позволяет объединять несколько итерируемых объектов (например, списки, кортежи) в один, создавая кортежи из элементов на соответствующих позициях.

Синтаксис



```
zip(*iterables)
```




***iterables**

Любые итерируемые объекты (списки, кортежи, строки и т.д.), которые будут объединены.

Примеры использования



Объединение нескольких итерируемых объектов одинаковой длины

```
names = ["Alice", "Bob", "Charlie"]
```

```
ages = [25, 30, 35]
```

```
cities = ['Hamburg', 'Berlin', 'Munich']
```

```
combined = zip(names, ages, cities)
```

```
print(combined)
```

```
print(list(combined))
```

Примеры использования



Объединение нескольких итерируемых объектов разной длины

```
list1 = [1, 2, 3]
```

```
list2 = ['a', 'b']
```

```
result = zip(list1, list2)
```

```
print(list(result))
```

Примеры использования



```
# Использование в цикле for

names = ["Alice", "Bob", "Charlie"]

ages = [25, 30, 35]

for name, age in zip(names, ages):
    print(f"{name} is {age} years old.")
```

Экспресс-опрос



Что такое стек и очередь?



Стек и очередь

Это структуры данных, используемые для хранения элементов с определёнными правилами доступа и управления.

Стек



Стек (Stack) работает по принципу **LIFO (Last In, First Out)**, что означает «последним пришёл — первым ушёл». Элементы добавляются и удаляются с одного конца, называемого вершиной стека.

Стек



Примеры

- История браузера (возврат на предыдущие страницы).
- Операции отмены (**Ctrl+Z**) в текстовых редакторах.

Операции

- Добавление элемента в вершину стека.
- Удаление элемента из вершины стека.

Список для реализации стека



```
stack = []

# Добавление элементов в стек

stack.append(1)

stack.append(2)

stack.append(3)

stack.append(4)

# Удаление последних элементов из стека

print(stack.pop())

print(stack.pop())

# Текущий стек

print(stack)
```

Очередь



Очередь (Queue) работает по принципу **FIFO (First In, First Out)**, что означает «первым пришёл — первым ушёл». Элементы добавляются в один конец очереди (в хвост) и удаляются с другого конца (с головы).

Очередь



Примеры

- Очередь задач в принтере.
- Обработка запросов в серверных системах.

Операции

- Добавление элемента в конец очереди.
- Удаление элемента с начала очереди.

Очередь



Для очереди рекомендуется использовать `deque` из модуля `collections` для повышения производительности.

Очередь



```
from collections import deque
```

```
queue = deque()
```

```
# Добавление элементов в очередь
```

```
queue.append(1)
```

```
queue.append(2)
```

```
queue.append(3)
```

```
queue.append(4)
```

```
# Удаление первых элементов из очереди
```

```
print(queue.popleft())
```

```
print(queue.popleft())
```

```
# Текущая очередь
```

```
print(queue)
```

Экспресс-опрос

?

Что такое устойчивость сортировки?



Устойчивость сортировки

Это свойство алгоритма сортировки сохранять относительный порядок элементов с одинаковыми значениями в исходной последовательности.

Пример устойчивости сортировки



```
words = ["orange", "mango", "apple", "banana", "kiwi", "cherry"]
```

```
# Сортировка списка по длине строк
```

```
sorted_words = sorted(words, key=len)
```

```
for word in sorted_words:
```

```
    print(f"{len(word)}: {word}")
```


Практическое применение устойчивости



Устойчивость сортировки важна в ситуациях, когда нужно сохранять исходный порядок элементов с одинаковыми значениями для последующей обработки данных.



ВОПРОСЫ



Домашнее задание

Одно слово

Напишите программу, которая обрабатывает список из строк и удаляет все строки, содержащие более одного слова, а также преобразует оставшиеся строки к нижнему регистру.

Данные:

```
text_list = ["Hello", "Python Programming", "World", "Advanced Topics", "Simple"]
```

Пример вывода:

Обработанный список: ['hello', 'world', 'simple']

Домашнее задание

Обновление цен товаров

Дан список товаров с ценами. Программа должна применить скидку к каждому товару и добавить в список элемент с новой ценой. В конце вывести таблицу с новой ценой.

Данные:

```
products = [{"Laptop", 1200}, {"Mouse", 25}, {"Keyboard", 75}, {"Monitor", 200}]
```

Пример вывода:

Введите скидку (в процентах): 17

Товар	Старая цена	Новая цена
Laptop	1200.00\$	996.00\$
Mouse	25.00\$	20.75\$
Keyboard	75.00\$	62.25\$
Monitor	200.00\$	166.00\$

Домашнее задание

Оценки текстом

Напишите программу, которая преобразует список оценок по системе от 1 до 5 в текстовое представление. Нужно сохранить в списках числовой результат и текстовое представление. Где, 5 — "отлично", 3-4 — "хорошо", а 2 и ниже — "неудовлетворительно".

Данные:

```
grades = [5, 3, 4, 2, 1, 5, 3]
```

Домашнее задание

Правильные скобки

Напишите программу, которая принимает строку, содержащую любые виды скобок — круглые `()`, квадратные `[]` и фигурные `{}`.

Необходимо проверить, правильно ли они расставлены. Скобки считаются правильно расставленными, если:

1. Каждая открывающая скобка имеет соответствующую закрывающую.
2. Скобки закрываются в правильном порядке.

Пример данных:

```
string = "({[]})"
```

Заключение

